

AstroBurst

High-Performance Astronomical Image Processor

Technical Document — Release v0.6.0

Samuel Krieger Bonini

September 2026

Field	Value
Document Version	0.6.0
Release	v0.6.0
Date	September 2026
Author	Samuel Krieger Bonini
Stack	Rust 1.75+ · Tauri v2.10 · React 19 · TypeScript 5.7
Platforms	Windows · macOS · Linux
License	AGPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1	Executive Summary	4
1.1	What's New in v0.6.0	4
1.2	What's New in v0.5.8	7
1.3	What's New in v0.5.5	9
1.4	What's New in v0.4.6	9
1.5	What's New in v0.4.5	10
1.6	What's New in v0.4.2	10
1.7	What's New in v0.4.0	10
2	System Architecture	11
2.1	Backend (Rust)	11
2.2	Non-Destructive Composite Pipeline (v0.4.5)	12
2.3	Frontend (React + TypeScript)	12
2.4	IPC Strategy	13

3	Mathematical Foundations	13
3.1	Screen Transfer Function (STF)	13
3.2	Asinh Stretch	14
3.3	Masked Stretch (v0.4.2)	14
3.4	Cross-Correlation Alignment (Phase Correlation)	14
3.5	Star-Based Affine Alignment (v0.4.0)	14
3.6	Sigma-Clipped Stacking	15
3.7	Channel Blending (v0.4.2)	15
3.8	White Balance (v0.4.0, non-destructive v0.4.5)	15
3.9	SCNR (v0.4.0, non-destructive v0.4.5)	15
3.10	Drizzle Integration	16
3.11	Richardson-Lucy Deconvolution	16
3.12	Background Extraction	16
3.13	Wavelet Denoising	16
3.14	Calibration Pipeline	16
3.15	Star Detection	16
3.16	SPCC (Spectrophotometric Color Calibration, v0.4.2)	17
3.17	Pixel Rejection Families (v0.6.0)	17
3.18	Frame Normalization (v0.6.0)	18
3.19	Thin-Plate Spline Background Extraction — DBE (v0.6.0)	19
3.20	Aperture Photometry with Fractional Pixel Weights (v0.6.0)	20
3.21	Barycentric and Heliocentric Radial Velocity (v0.6.0)	22
3.22	Moment Maps (v0.6.0)	23
3.23	Wavelet Per-Scale Detail Bias (v0.6.0)	24
3.24	Local Histogram Equalization and HDR Multiscale Transform (v0.6.0)	25
4	Command Module Reference	26
5	FITS and ASDF I/O	27
5.1	Image References (v0.6.0)	28
5.2	Lossless Integer Planes (v0.6.0)	28
5.3	Multi-Extension Output (v0.6.0)	28
6	WebGPU Compute Pipeline	28
6.1	Display Path Unification (v0.6.0)	28
7	Headless HTTP Server	29
7.1	Session Model	29

7.2	The v2 API Surface	31
7.3	Job Model	32
7.4	What Is Shared, and What Is Not	32
7.5	Error Model	33
8	Performance Benchmarks	34
9	Dependencies	34
10	Known Limitations	36
11	Roadmap	37
A	Sample Test Data	38

1 Executive Summary

AstroBurst is a cross-platform desktop application for processing, analyzing, and visualizing astronomical FITS and ASDF images. It is the first astronomical image processor built on the **Rust/Tauri/WebGPU** stack, combining native-level performance with a modern, accessible user interface.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Version 0.6.0 delivers a complete processing pipeline from raw FITS/ASDF ingestion through calibration, stacking, alignment, color composition, and export, with **107 backend commands** exposed through a typed TypeScript API layer and a 10-step ComposeWizard pipeline. Beyond the desktop application, the same processing core is exposed over HTTP by an optional headless server (Section 7), so region statistics, rendering, histograms and cutouts are reachable from a script without a display.

The releases between 0.4.6 and 0.6.0 moved the project from a composition tool towards an instrument-aware one: any FITS HDU or ASDF array can be displayed and measured, data-quality flags are decoded against per-telescope bit tables, photometry is calibrated from header zero points into AB magnitudes and janskys, and spectral cubes carry a real FITS spectral axis with velocity conventions and barycentric correction.

1.1 What's New in v0.6.0

Version 0.6.0 is published as a preview of the 0.6 line: the viewer maturity phases (0 to 3), a processing-parity round (Phase 4) and a science-analysis round (Phase 5), together with the fixes raised by their reviews. The Windows MSI bundle reports 0.6.0 rather than 0.6.0-preview, because MSI version numbers cannot carry a pre-release identifier.

- **Phase 0+1 (viewer maturity)**
 - **Display controls** — Scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`), limit modes (`minmax`, `zscale`, `percentile`, `user`) and 9 colormaps plus invert; the GPU path samples a 256×1 LUT texture and the worker fallback reproduces the same bytes, with stretch, limits and the byte rule defined once in `core/imaging/scale.rs` and shared by `compute_scale_limits_cmd`, `get_colormap_lut_cmd` and the server
 - **Pixel readout** — `probe_pixel_cmd` reports value, unit (FITS BUNIT or ASDF unit), box min/max/mean/median and pixel/NaN counts from `core/imaging/pixel_probe.rs`, shared with the server `/v2/.../pixel` endpoint
 - **Coordinate frames** — ICRS, FK5 (J2000), galactic and ecliptic (J2000) cursor readout in sexagesimal or decimal form, via `core/astrometry/frames.rs` and the `frame` argument of `pixel_to_world_cmd`
 - **Test CI** — `backend-tests-unix` matrix (Ubuntu/macOS, `cargo test -all-features`) and a `frontend-tests` job; `vitest` added with `pnpm test`, covering `pixelMapping`, `filterWavelengths` and the binary IPC parsers
- **Phase 2 (planes, DQ)**
 - **Image references** — Every path-based command accepts `<path>#hdu=<n>` (any FITS HDU) or `<path>#array=<key>` (any ASDF array, dotted keys such as `roman.dq`); the reference is the cache key and output stems derive from it. Parsing lives in `types/image_ref.rs`

- and a single loader (`infra/image_source.rs`) serves both the desktop commands and the server
- **Lossless integer planes** — BITPIX 8/16/32 HDUs and int8–int32, uint8–uint32 and bool8 ASDF arrays decode bit-exactly (`IntPlane`, with BZERO 32768/2147483648 handled without float rounding) alongside the float view, cached next to the pixels and counted in the cache budget
 - **DQ flag tables** — JWST (32 bits), HST (15 bits) and Roman conventions selected from `TELESCOP/INSTRUME` or the ASDF `meta.telescope/meta.instrument.name` cards; `get_dq_flag_table_cmd` returns the flags, the default overlay mask and the exclusion mask
 - **Companion readout** — DQ and ERR planes resolved by `EXTNAME+EXTVER` (FITS) or sibling array keys (ASDF); the pixel probe and the server endpoint add a `dq` block (bits, value, names, table, decoded text) and an `err` block (value, unit)
 - **DQ overlay** — `get_dq_mask_preview` returns a binary OR-reduced mask grid with a 16-byte little-endian header (width, height, mask, table id) on the same grid as `get_raw_pixels_preview`, both sharing `preview_dims/cell_range`
 - **DQ-masked statistics** — `compute_histogram(excludeDq)` and `measure_photometry_cmd(excludeDq)` treat `DO_NOT_USE` (JWST convention) or the documented HST bad-pixel bits as NaN and report `masked/dq_excluded` counts
 - **Phase 3 (regions)**
 - **Interactive regions** — Circle, ellipse, box, annulus, polygon, line and point drawn on a dedicated overlay layer, with selection, move and resize handles, rotation for box and ellipse, `Delete` to remove and `Escape` to cancel; the tool returns to select once a shape is committed
 - **DS9 region files** — `regions_import_cmd/regions_export_cmd` read and write `.reg` in the `image`, `icrs` and `fk5` systems (degrees or sexagesimal, radii in pixels or arc-sec/arcmin/degree, `global` and `local` properties); sky regions convert through the WCS
 - **Per-region statistics** — `region_stats_cmd` returns count, sum, mean, median, MAD, sigma, min, max and sigma-clipped mean/median, with background annulus subtraction yielding net sum and SNR; DQ-excluded pixels are reported separately
 - **Profiles and cuts** — `radial_profile_cmd` (mean, median, standard deviation and cumulative sum per integer radius, optional background annulus) and `line_cut_cmd` (bilinear samples along a line), plotted on canvas in the Analysis tab
 - **Shared geometry** — Region geometry lives once in `core/imaging/region.rs` under the DS9 convention (a pixel belongs to a region when its centre is inside, so a radius-5 circle on an integer centre covers 81 pixels); the server `v2` region, stats and cutout endpoints use the same shapes
 - **Phase 4 (processing parity)**
 - **Pixel rejection** — Sigma clipping, Winsorized sigma clipping, linear fit clipping, percentile clipping, min/max or none, combined by mean, median, min or max; selectable in the Stack tab, the compose wizard, the calibration pipeline and the headless server, with unknown names rejected as 400
 - **Frame normalization** — None, additive, multiplicative and either with scaling, computed on the pixels finite in every frame, plus `scale+offset` rejection normalization and optional `_rejection_low/_rejection_high` maps; the rejection kernel lives once in `core/stacking/combine.rs` and the batch pipeline uses it
 - **Master frames** — Bias, dark and flat masters are integrated with that same kernel (`MasterConfig`, Winsorized sigma clipping 4/3 and averaging by default, flats scaled to the first frame’s median) instead of a plain per-pixel median
 - **Cosmetic correction** — Hot/cold pixels from a master dark with separate sigma

- thresholds, auto-detection against the local median with star-core protection, PixInsight-style defect lists (`Point`, `Col`, `Row`, 0-based) validated line by line, CFA-aware replacement by the median or mean of unflagged neighbours blended by an amount, and batch mode over loaded lights; in the pipeline the step runs after dark subtraction and before flat division
- **Spline (DBE) background** — A regularised thin-plate spline through box samples with automatic grid placement, `Point` regions as manual samples, star and outlier rejection, adjustable smoothing, subtract/divide modes and median-preserving normalization; writes `_dbe_corrected.fits` and `_dbe_model.fits` with the source WCS and an ABPROC provenance card
 - **PixelMath** — A per-pixel expression language over the target image (`$T`) and named image slots, with arithmetic, comparison and logical operators, `iif`, inversion, per-pixel functions and image statistics over finite pixels; NaN-preserving, with optional truncate or rescale to $[0, 1]$, output to `_pixelmath.fits` carrying ABPROC and PMEXPR cards, and live validation with a caret on the failing token
 - **Local Histogram Equalization** — CLAHE with circular or square kernel, contrast limit, 8/10/12-bit histograms and blend amount, applied on luminance so hue is preserved
 - **HDR Multiscale Transform** — 2–8 wavelet layers with iterations, overdrive, inverted mode, lightness or per-channel operation and star-core deringing; refuses linear input and states that a stretch must come first
 - **Multiscale API** — Wavelet noise reduction gains an MLT-style per-scale detail bias (-1 to $+3$); the à-trous decomposition (`atrous_decompose`, `atrous_reconstruct`, `atrous_reconstruct_with_bias`) and the noise estimators (`noise_sigma_mad`, iterative `k_sigma_noise`) are public for other multiscale tools
 - **Statistics and noise** — An exact statistics table (count, count %, mean, median, avgDev, MAD, $\sqrt{\text{BWMV}}$, stdDev, variance, min, max, sum, NaN, excluded) in raw, $[0, 1]$ or 16-bit units with no histogram approximation, per channel for composites and with region/DQ options; k-sigma multiresolution noise is exposed in the panel, on `POST /v2/sessions/:sid/stats`, in `evaluate_noise_batch_cmd`, as a Subframe Selector column and as a $1/\sigma^2$ frame-weighting toggle in the Stack tab
- **Phase 5 (science analysis)**
 - **Photometric calibration** — `core/metadata/photcal.rs` reads JWST MJy/sr (via `PIXAR_SR` or a WCS-derived pixel area, flagged as derived) and DN/s via `PHOTMJSR`, HST `PHOTFLAM`/`PHOTPLAM`/`PHOTZPT` (STmag and ABmag), Roman `conversion_megajanskys/pixelarea_sterra` and generic zero-point cards; the Photometry panel reports AB magnitude with error, flux in Jy, surface brightness and the convention used
 - **Aperture photometry** — Fractional edge-pixel weights in the shared `core/analysis/aperture.rs` (5×5 sub-pixels by default), ERR-plane error propagation or sky noise with an optional Poisson term, the SATURATED DQ bit, masked-pixel counts and a curve-of-growth aperture correction with the total magnitude; region statistics on images with an ERR plane report the summed error and the inverse-variance weighted mean
 - **FITS spectral axis** — `core/astrometry/spectral.rs` handles `WAVE`/`AWAV`/`FREQ`/`VRAD`/`VOPT` axes with `CDEL3`, `CD3_3` or `PC3_3` steps, lenient `CUNIT3` spellings with FITS default units, `RESTWAV`/`RESTFRQ`, `SPECSYS`/`VELOSYS` and explicit errors for `-LOG` and `-TAB` axes, so MUSE `CD3_3` and ALMA `FREQ` cubes get a correct axis
 - **Velocity and barycentric correction** — Air/vacuum conversion (Greisen et al. 2006, FITS Paper III eq. 65), optical, radio and relativistic velocity axes from a rest wavelength, and a barycentric/heliocentric radial-velocity correction for ground-based headers (site and mid-exposure recovered from the usual cards, Meeus low-precision ephemeris, about 0.02 km/s), with FITS date, JD/MJD and GMST helpers

- in `core/astrometry/time.rs` and panel controls backed by `spectral_axis_cmd`, `velocity_axis_cmd` and `radial_velocity_correction_cmd`
- **Spectral-cube science** — Region spectra over circle, box, ellipse or polygon with optional annulus sky subtraction, in native units and in Jy for MJy/sr cubes (`get_cube_spectrum_region_` channel-range collapse by sum, mean or median saved as 2D FITS with the spectral axis removed (`collapse_cube_range_cmd`); and M0, M1, M2 moment maps with a per-pixel continuum fit from two side windows, SNR masking and a velocity axis (`moment_maps_cmd`)
 - **Science cutout export** — A multi-extension FITS (SCI, ERR, DQ) from a box region or a manual centre and size in pixels or arcseconds, with CRPIX shifted, LTV1/LTV2/LTM1_1/LTM2_2 recorded, NaN padding outside the parent and padded DQ pixels flagged; `core/imaging/cutout.rs` is shared with the server endpoint, and the FITS writer gains `write_mef_images` for plain multi-HDU IMAGE files
 - **WCS coordinate grid** — ICRS, FK5, galactic or ecliptic iso-lines traced in world coordinates with sexagesimal-friendly steps, RA 0h wrap and polar fields handled and non-overlapping edge labels; `grid_lines_cmd` and `POST /v2/sessions/:sid/wcs/grid` return the polylines, labels and steps, drawn through a non-persisted overlay store (`src/utils/overlayStore.ts`) shared by the CPU and GPU viewers
 - **Gaia DR3 catalog panel** — Vizier cone search around the image centre with proper motions propagated from J2016.0 to the observation date, rows placed on the image with labels, full-field cross-match of detected stars with astrometric residuals (median dRA/dDec, rms), a 3-sigma-clipped photometric zero point in G, BP or RP with an optional colour term, RFC-4180 CSV export and one-click conversion of rows into Point regions
 - **Search Everywhere** — IntelliJ-style command palette (Ctrl+K or double Shift) with fuzzy search over processed files, tool panels and actions, full keyboard navigation and a status-bar entry point
 - **Workspace layout** — Resizable right tool panel (280–640 px), zero-width splitters with a 7px grab area and double-click reset, animated tool windows (200 ms slide, hidden panels inert and unmounted), prefers-reduced-motion support, and splitter positions persisted through `src/utils/layout.ts`
 - **WCS engine** — Replaced with the `wcs` crate (`wcs-rs` + `mapproj`), taking projection support from TAN/SIN/ARC/CAR to roughly 20 FITS projections and adding proper CD/PC/CDELTA matrix handling; `WcsTransform`'s public API is unchanged, and `pixel_to_world_cmd` replaces the duplicated client-side implementation that was removed
 - **SIP is evaluated by AstroBurst** — `mapproj` 0.4.0's SIP evaluator advances polynomial powers by repeated squaring instead of by degree, silently producing wrong sky coordinates for any SIP header; the wrapper strips the `-SIP CTYPE` suffix before constructing the `wcs-rs` engine so that path never runs, and applies and inverts SIP with AstroBurst's own math
 - **GPU viewer parity with processing** — The processed FITS now becomes the raw-pixel source of the WebGPU renderer for every mono step, with histogram and auto-STF recomputed for it, instead of the GPU path keeping the original pixels while only the CPU path switched to the rendered PNG; Reset returns both paths to the original image

1.2 What's New in v0.5.8

Version 0.5.8 is an audit release: the math, GPU flow, image-processing and performance paths were reviewed across 21 module groups with every finding reproduced by hand before being fixed, and the ASDF reader was validated against the ASDF Standard 1.5/1.6.

- **Drizzle kernel** — The Lanczos3 kernel was evaluated in output-pixel units, so at scale 3 (and any integer scale with zero offsets) 8/9 of the output pixels received zero weight and rendered as holes; the kernel argument and support now scale with the output grid
- **Sigma-clip floor** — Sigma-clip combine and the batch pipeline used a $1e-10$ sigma floor when the MAD is 0 (more than half the samples tied), rejecting every value that differed from the median; they now fall back to the mean absolute deviation and skip rejection when the scale is truly zero
- **Star removal** — The soft star mask was applied twice, since push-pull fill already blends by the mask, leaving $m - m^2$ of the star flux at every mask edge
- **GHS stretch** — $D = 0$ returned the raw, un-normalized data instead of the normalized identity, producing a white preview and export once the composite was cached
- **Wavelet reconstruction** — Negative pixels were clamped to 0 and NaN was rewritten as 0
- **Phase correlation** — A confident coarse shift was discarded whenever the fixed centre crop was featureless, falling back to identity or noise; the coarse estimate is now kept when the refine pass has no signal
- **Star detection** — Saturated flat-top stars were deblended into several detections; equal-valued adjacent maxima now collapse to a single seed
- **GPU/CPU preview parity** — IPC preview extrema were computed over all finite pixels while auto-STF came from padding-aware statistics, so the GPU render and the canonical PNG disagreed; NaN pixels were rewritten to 0.0 before upload and rendered gray; and linked STF on the GPU normalized each channel by its own range while the Rust path uses the combined range
- **WebGPU recovery** — A failed initialisation (no adapter, device creation error) was cached for the whole session; the retry path re-probes
- **Compressed FITS tiles** — Quantized GZIP_1/GZIP_2 tiles were decoded as raw f32 bit patterns (the integer width is now inferred from the tile byte count), ZBITPIX ± 64 was rejected for every codec including AstroBurst's own GZIP_2 files, the Rice decoder indexed the stream unchecked and panicked on short or corrupt tiles, and constant tiles quantized with scale 1 under SUBTRACTIVE_DITHER_1 gained ± 0.5 noise on decode
- **ASDF block handling** — Blocks are memory-mapped and decompressed lazily, so only the block behind the selected array is decoded (Roman L2 files carry six or more full-frame arrays), and uncompressed contiguous arrays are borrowed instead of copied three times
- **ASDF conformance** — #ASDF BLOCK INDEX trailers, CRLF line endings, tree padding, STREAMED blocks and `shape: ['*', ...]` are handled per spec; negative strides were silently read as contiguous; external block references now fail explicitly; gWCS chains under `roman.meta.wcs` are resolved with the Shift offset converted from 0-based gwcs pixels to 1-based CRPIX; YAML-tagged meta subtrees are flattened into the header instead of dropped; and allocation sizes from untrusted block headers are bounded
- **Headless server correctness** — Region, histogram, cutout and pixel endpoints validated inputs only after arithmetic that could wrap or allocate unbounded memory; statistics and histograms silently discarded every pixel $\leq 1e-7$, giving wrong results for bias-subtracted or difference images; binning by a non-divisible factor averaged overlapping windows; viewport auto-STF from crop statistics was applied with global normalization; and re-opening an image under an existing name returned the stale cached array
- **Job state** — Cancellation is observed by workers before results are published and terminal transitions are atomic, so a late worker can no longer overwrite `cancelled`
- **Memory and throughput** — The batch stacking pipeline loads and stacks one channel at a time and returns 2048 px previews instead of full-resolution base64 masters; drizzle scatters contributions directly into per-band accumulators in parallel instead of materialising every contribution as a tuple; and cube global statistics sample at most 32 frames with

a pixel stride bounded to roughly 8M samples

1.3 What's New in v0.5.5

- **WebGPU RGB preview** — GPU-only `GpuRgbRenderer` renders full RGB composites from three `r32float` textures with per-channel STF in the WGS� shader (MTF identical to the mono shader, the CPU worker and the Rust backend); falls back to the PNG composite when WebGPU is unavailable, no adapter is found or the device is lost
- **Live per-channel RGB STF** — `RgbStfPanel` in the Analysis tab restretches the GPU composite live with no backend round-trip; the mono STF slider skips its backend render while a GPU preview is active
- **Linear RGB preview command** — `get_raw_rgb_pixels_preview(path?, max_dim)` returns a linear 3-channel float preview with per-channel min/max from a native RGB FITS or the composite cache (planar R|G|B behind a 32-byte header), with `encode_channel_preview` factored out of the mono encoder byte-identically
- **Canonical filter table** — `FILTER_WAVELENGTHS_NM` and `filter_to_wavelength_nm` in `types/constants.rs`, mirrored by `utils/filterWavelengths.ts`, normalize CLEAR, separators and dual filters, replacing two duplicated JWST tables
- **Dynamic channel Auto-Map** — Each non-narrowband, non-RGB filter gets its own wavelength-labelled channel grouped by nm, so NIRCā F090W–F444W yields five distinct channels
- **Auto (λ) blend** — Spreads any number of channels across R/G/B by wavelength, applied once for untouched broadband stacks and never overriding a palette or manual weights
- **Preview downsampling** — Nearest-neighbour replaced by a contrast-weighted hybrid (mean to max) area kernel that preserves faint point sources while anti-aliasing the background
- **STF GPU/CPU parity** — The CPU fallback worker no longer hard-zeros raw $\leq 1e-7$, which blacked out background-subtracted pixels that the GPU shader maps to gray; epsilons are aligned at $1e-8$
- **Preview normalization** — `data_min/data_max` describe the full image through a shared `scan_extrema` instead of the decimated subset, so the live preview matches the canonical PNG
- **GPU failure handling** — `GpuSingleton` exposes `onGpuLost` and an uncaptured-error handler so a dead device's resources are dropped and the CPU worker takes over, an oversized image degrades to the worker instead of a blank canvas, and switching files no longer leaves stale pixels blocking the fetch
- **Dead code removed** — The orphaned ZNCC WebGPU shader (`zncc_align.wgsl`), the unreachable client-side downsample worker and unused imports

1.4 What's New in v0.4.6

- **Tone curves (AdjustStep)** — Spline-based curve editor with per-channel R/G/B and linked RGB modes; non-destructive (reads `COMPOSITE_KEY`, applies STF + curves for preview only)
- **Auto-STF preview** — All linear-domain commands (blend, calibrate, WB, SCNR, reset) render preview with linked auto-STF, eliminating washed-out previews
- **Cache-only intermediate processing** — align, stack, calibrate, and background extraction store results in `GLOBAL_IMAGE_CACHE`; only PNG previews written to disk until explicit export
- **Narrowband detection via blend preset** — `isNarrowbandWorkflow` checks preset (SHO, HOO, Foraxx) as fallback when bin IDs are r/g/b; SCNR warning badge for narrowband

workflows

- **Auto-STF propagation** — blend, calibrate, WB, and reset commands return `auto_stf` in response; StretchStep sliders initialize from auto-STF; ColorBalanceStep propagates post-WB STF to CompositeContext
- **Export pipeline fixes** — STF cache hit corruption fixed (linear data rendered directly when `applyStfStretch = false`); WCS header preservation for composite FITS; sentinel path handling
- **Star detection safety** — NaN guard (`is_finite && > 1e-7`), degenerate image early return, NaN-safe sort comparator
- **TypeScript strict mode** — 39 errors resolved (0 remaining); 22 missing lucide-react declarations added
- **Windows asset protocol** — Path safety with canonicalize, URL decode compatibility, async runtime
- **Flatpak/Flathub** — Submission pending approval for Linux distribution

1.5 What's New in v0.4.5

- **Non-destructive composite pipeline** — Immutable ORIG cache with idempotent WB and SCNR; `reset_wb_cmd` for one-click undo
- **STF rendering consistency** — GPU shader, CPU worker, and Rust backend produce pixel-identical output via symmetric denominator protection and unified padding threshold
- **Blend preset positional fallback** — SHO/HOO/Foraxx presets work with any bin configuration via positional weight mapping
- **Spectroscopy wavelength unit conversion** — Automatic CUNIT3 detection (M, NM, ANGSTROM, HZ) with display conversion
- **Export pipeline fix** — Exports use user STF params instead of identity, matching preview

1.6 What's New in v0.4.2

- **ComposeWizard** — 11-step pipeline (Channels, Stack, Align, Crop, BG, Blend, Color, Mask, Stretch, Adjust, Export)
- **Synthetic FITS generation** — Star field, PSF, and noise injection for testing
- **Narrowband palette system** — SHO, HOO, HOS, NaturalColor, Custom with auto-detection
- **Live composite STF re-stretch** — Per-channel shadow/midtone/highlight without recomposing
- **SPCC** — Spectrophotometric color calibration with optional Gaia DR3 TAP (vizier feature)
- **Masked stretch** — Iterative MTF with star mask protection
- **PSF enhancement** — Moment-based FWHM with subpixel peak estimation

1.7 What's New in v0.4.0

- **Star-based affine alignment** — Triangle asterism matching with RANSAC, automatic fallback to FFT phase correlation
- **Stability-based auto white balance** — Lowest MAD/median channel as reference
- **SCNR luminance redistribution** — ITU-R BT.709 weighted redistribution to R and B
- **Numerical correctness audit** — NaN ordering, Hann window, polynomial basis, RL regularization, phase correlation confidence

2 System Architecture

AstroBurst follows a strict backend/frontend separation, with all computation offloaded to Rust and all rendering handled by the browser engine embedded in Tauri.

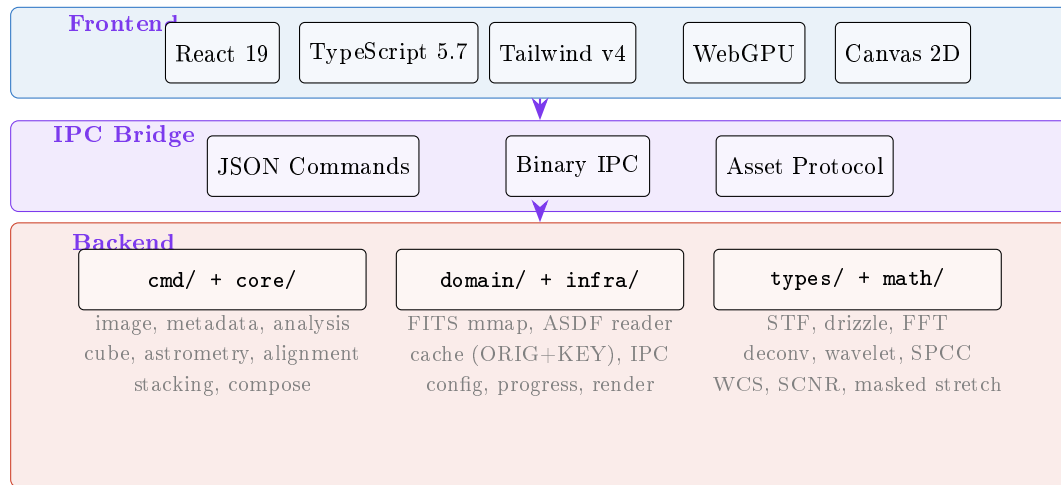


Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The Rust backend is organized into six layers:

<code>cmd/</code>	Tauri command handlers organized by domain (47 registered commands). Each module receives deserialized JSON arguments, delegates to <code>core/</code> / <code>domain/</code> logic, and returns JSON or binary responses.
<code>core/</code>	Pure computational modules: alignment (phase correlation, affine with triangle asterism matching), analysis (histogram, FFT, stars), astrometry (plate solve, WCS, SPCC), compose (RGB, channel blend, SCNR, white balance, LRGB), cube (spectrum, frames), imaging (STF, deconvolution, wavelet, background, masked stretch, PSF estimation, star mask), metadata (header discovery, filter detection), stacking (sigma-clip, drizzle), synth (star field, PSF, noise).
<code>domain/</code>	Orchestration layer coordinating core modules for complex multi-step operations (calibration masters, drizzle-from-paths, cube processing, plate solve).
<code>infra/</code>	Infrastructure: Arc-based zero-copy cache with ORIG/KEY dual-layer for non-destructive composite editing, config management, FITS I/O (mmap parser, MEF scanner, FITS writer with multi-BITPIX), ASDF reader (YAML tree parser, binary block extractor, zlib/bzip2/lz4 decompressor, Roman data model converter, gWCS bridge), binary IPC encoding, progress tracking, PNG rendering (8/16-bit grayscale and RGB).
<code>types/</code>	Shared type definitions: image stats, STF params, stacking/drizzle/RL configs, compose types (AlignMethod, WhiteBalance, BlendWeight), SCNR, error types, FITS header abstraction, 310+ constants.
<code>math/</code>	Mathematical primitives: NaN-safe median/MAD, sigma clipping, SIMD log approximation (Cephes degree-8 minimax, < 1 ULP precision).

2.2 Non-Destructive Composite Pipeline (v0.4.5)

The composite cache uses a dual-layer architecture:

ORIG/KEY Cache Architecture		
Layer	Keys	Semantics
ORIG (immutable)	<code>--composite_orig_r/g/b</code>	Written once by <code>blend/compose/RGB-FITS load</code>
KEY (working)	<code>--composite_r/g/b</code>	Reconstructed: $\text{ORIG} \times \text{WB} \rightarrow \text{KEY}$, or $\text{ORIG} \times \text{WB} \rightarrow$

Every downstream operation (`calibrate_composite_cmd`, `apply_scnr_cmd`) reads from ORIG, applies the full transform chain, and writes to KEY. This makes WB and SCNR idempotent: applying the same factors twice produces identical results. `reset_wb_cmd` copies `ORIG` $\rightarrow$ `KEY`.

2.3 Frontend (React + TypeScript)

The frontend is organized around the ComposeWizard (10-step pipeline) and split preview contexts:

services/ 24 domain-specific service modules wrapping Tauri IPC: analysis, astrometry, catalog, compose, config, cosmetic, cube, cutout, db, display, export, fits, header, localContrast, pixelmath, processing, progress, regions, spectral, stacking, statistics, synth, tiles, tone. Command name strings live only in these modules; components never call `typedInvoke` directly.

ComposeWizard

10-step pipeline: Channels $\rightarrow$ Stack $\rightarrow$ Align $\rightarrow$ Crop $\rightarrow$ Background $\rightarrow$ Blend $\rightarrow$ Colour $\rightarrow$ Stretch $\rightarrow$ Adjust $\rightarrow$ Export. The separate Mask step of v0.4.2 was folded into Stretch, where star protection is now a mode of the masked stretch rather than a stage of its own. Internal `useReducer` state with step completion tracking and suggested-next-step logic; each step declares an `enabled` predicate over the wizard data, so availability is derived from state rather than from a completion flag.

Preview Contexts

12 split React contexts (FileCtx, DisplayCtx, DoneFilesCtx, HistCtx, CubeCtx, RgbCtx, RenderCtx, RenderActionsCtx, StarOverlayCtx, RawPixelsCtx, NarrowbandCtx, DqCtx) for granular re-render control. Splitting them this finely is what keeps a cursor move over the image from re-rendering the file list or the wizard.

GPU Rendering

WebGPU singleton with WGSL fragment shader for real-time STF. CPU worker fallback with identical MTF implementation. Both use symmetric denominator protection and padding threshold matching the Rust backend.

2.4 IPC Strategy

Binary IPC Protocol

The `get_raw_pixels_preview` command encodes a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	u32	4B	Image width
4	u32	4B	Image height
8	f32	4B	Data minimum
12	f32	4B	Data maximum
16	f32[]	$W \times H \times 4B$	Raw pixel values

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\text{MTF}(x, m) = \frac{(m - 1) \cdot x}{(2m - 1) \cdot x - m} \quad (1)$$

Properties:

- $\text{MTF}(0, m) = 0$ and $\text{MTF}(1, m) = 1$ for all m
- $\text{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it
- When $m = 0.5$: $\text{MTF}(x, 0.5) = x$ (identity)

Denominator Protection (v0.4.5)

The denominator $D = (2m - 1)x - m$ changes sign across the valid domain. The correct guard is:

$$\text{if } |D| < \varepsilon \text{ then return } x; \quad \text{if } |m - 0.5| < \varepsilon \text{ then return } x$$

The v0.4.2 GPU shader incorrectly used $\max(D, \varepsilon)$ which forces $D > 0$, inverting the transfer function for $m < 0.5$. v0.4.5 uses symmetric $|D|$ protection across GPU (WGSL), CPU worker (JS), and Rust backend, achieving pixel-identical output.

Padding pixels ($v \leq 10^{-7}$) are rendered as black across all three rendering paths, matching the Rust `is_valid_pixel` threshold.

The **auto-stretch** computes parameters from image statistics:

$$\text{shadow} = \max(0, \tilde{x}_{\text{norm}} + k \cdot \sigma_{\text{norm}}), \quad k = -2.8 \quad (2)$$

$$\text{midtone} = \text{MTF}_{\text{balance}}^{-1} \left(\frac{\tilde{x}_{\text{norm}} - \text{shadow}}{1 - \text{shadow}}, t_{\text{bg}} \right), \quad t_{\text{bg}} = 0.25 \quad (3)$$

where $\tilde{x}_{\text{norm}}$ and σ_{norm} are the normalized median and sigma ($\text{MAD} \times 1.4826$).

3.2 Asinh Stretch

$$f(x) = \frac{\text{asinh}(\alpha \cdot x)}{\text{asinh}(\alpha)} \quad (4)$$

Maps $[0, 1] \rightarrow [0, 1]$, approximately linear for small x (preserving faint detail), logarithmic compression for bright values. $\alpha \in [1, 500]$ controls aggressiveness.

3.3 Masked Stretch (v0.4.2)

Iterative MTF stretch with star mask protection. At each iteration k :

$$M^{(k)} = \text{StarMask}(I^{(k)}, \text{growth}, \text{softness}) \quad (5)$$

$$I^{(k+1)} = (1 - \alpha \cdot M^{(k)}) \cdot \text{MTF}(I^{(k)}) + \alpha \cdot M^{(k)} \cdot I^{(k)} \quad (6)$$

where α is the protection amount (default 0.85) and growth controls the mask dilation radius. Convergence when background level reaches target (default 0.25). JWST-optimized defaults: growth 0.8, protection 96%.

3.4 Cross-Correlation Alignment (Phase Correlation)

$$\text{shift} = \arg \max_{(\Delta x, \Delta y)} \mathcal{F}^{-1} \left\{ \frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|} \right\} \quad (7)$$

Hann window: $w(n) = 0.5(1 - \cos(2\pi n/N))$. Confidence: $z = (P_{\text{peak}} - \mu_P)/\sigma_P$. Coarse-to-fine: downsample to 512×512 , refine on centered crop.

3.5 Star-Based Affine Alignment (v0.4.0)

Triangle descriptor (scale-invariant):

$$T = \left(\frac{d_{\text{mid}}}{d_{\text{short}}}, \frac{d_{\text{long}}}{d_{\text{short}}} \right), \quad \text{match if } |T_r - T_t| < 0.005 \quad (8)$$

RANSAC (500 iterations) fits 6-DOF affine or 4-DOF rigid:

$$\begin{pmatrix} x_t \\ y_t \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \quad (9)$$

Sanity: offset $< 25\%$, rotation $< 5^\circ$, scale $\in [0.85, 1.15]$, residual < 3 px, inlier ratio $> 30\%$. Fallback: affine $\rightarrow$ rigid $\rightarrow$ phase correlation $\rightarrow$ identity. Warp via bicubic Catmull-Rom, parallelized per row.

3.6 Sigma-Clipped Stacking

Algorithm 1: Sigma-Clipped Stacking (median+MAD first iter, mean+std subsequent)

Input: Frames $\{F_1, \dots, F_N\}$, $\sigma_{\text{low}}, \sigma_{\text{high}}$, max iters K
Output: Stacked image S

```

1 foreach pixel  $(i, j)$  do
2    $V \leftarrow \{F_k(i, j)\}$ , iter 0:  $\mu = \text{median}(V)$ ,  $\sigma = \text{MAD}(V) \times 1.4826$ ;
3   iters 1+:  $\mu = \text{mean}(V)$ ,  $\sigma = \text{std}(V)$ ;
4   reject  $v \notin [\mu - \sigma_{\text{low}} \cdot \sigma, \mu + \sigma_{\text{high}} \cdot \sigma]$ ;
5    $S(i, j) \leftarrow \text{mean}(V)$ ;
6 end
```

3.7 Channel Blending (v0.4.2)

Given N input channels C_k and weight matrix W :

$$\text{Out}_c = \sum_{k=1}^N W_{k,c} \cdot C_k, \quad c \in \{R, G, B\} \quad (10)$$

Preset weights (e.g., SHO: $\text{SII} \rightarrow R$, $\text{H}\alpha \rightarrow G$, $\text{OIII} \rightarrow B$) use named channelId matching with **positional fallback** (v0.4.5): when preset channelIds don't match filled bins, weights map positionally to the first N filled bins.

3.8 White Balance (v0.4.0, non-destructive v0.4.5)

Reference channel selection by minimum coefficient of variation:

$$k^* = \arg \min_{k \in \{R, G, B\}} \frac{\text{MAD}_k}{\tilde{C}_k}; \quad w_j = \frac{\tilde{C}_{k^*}}{\tilde{C}_j}, \quad j \neq k^*; \quad w_{k^*} = 1 \quad (11)$$

In v0.4.5, WB is applied from ORIG cache: $\text{KEY}_c = \text{ORIG}_c \times w_c$. Idempotent.

3.9 SCNR (v0.4.0, non-destructive v0.4.5)

Two limit methods and optional luminance preservation:

$$G' = \min(G, \frac{R+B}{2}) \quad \text{or} \quad \min(G, \max(R, B)) \quad (12)$$

$$G_{\text{new}} = G + \alpha(G' - G), \quad \alpha \in [0, 1] \quad (13)$$

When luminance preservation is enabled (BT.709):

$$\Delta = \frac{0.7152 \cdot (G - G_{\text{new}})}{0.2126 + 0.0722}; \quad R' = \min(R + \Delta, 1); \quad B' = \min(B + \Delta, 1) \quad (14)$$

In v0.4.5, SCNR is applied from ORIG cache with WB pre-applied: $\text{ORIG} \times \text{WB} \rightarrow \text{SCNR} \rightarrow \text{KEY}$. Idempotent.

3.10 Drizzle Integration

$$O(i, j) = \frac{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p)} \quad (15)$$

Three kernels: square (top-hat), Gaussian, Lanczos-3. Flat-storage accumulator (v0.4.0): `Vec<f32>` + `Vec<u16>` replacing `Vec<Vec<f32>>`. Finalize uses median/MAD sigma clipping (Stetson 1987).

3.11 Richardson-Lucy Deconvolution

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h} \right) \cdot \frac{1}{1 + \lambda} \quad (16)$$

Tikhonov regularization as multiplicative damping ($\lambda = 0.001$ default). FFT-based convolution.

3.12 Background Extraction

$$B(x, y) = \sum_{i=0}^d \sum_{j=0}^{d-i} a_{ij} x^i y^j \quad (17)$$

Polynomial degree 1–5, grid 3–32, sigma-clipped sampling with `powi` exponentiation.

3.13 Wavelet Denoising

A trous wavelet decomposition: $I = \sum_{k=1}^N W_k + c_N$. Hard threshold: $W'_k = W_k$ if $|W_k| > \sigma_k t_k$, else 0. Up to 8 scales.

3.14 Calibration Pipeline

$$I_{\text{cal}} = \frac{I_{\text{raw}} - M_{\text{bias}} - r \cdot M_{\text{dark}}}{M_{\text{flat}} / \bar{M}_{\text{flat}}} \quad (18)$$

Masters via median combine. Flat normalized by mean. Negative pixels clamped to zero.

3.15 Star Detection

Tile-based background (parallelized). Connected-component analysis with flux-weighted centroid:

$$\bar{x}_C = \frac{\sum_{(i,j) \in C} i \cdot (I_{ij} - b)}{\sum_{(i,j) \in C} (I_{ij} - b)} \quad (19)$$

FWHM from second intensity moments (true **median** across detected stars, v0.4.6):

$$\text{FWHM} = 2\sqrt{2 \ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \quad (20)$$

3.16 SPCC (Spectrophotometric Color Calibration, v0.4.2)

Color correction factors derived from stellar Bp-Rp color indices matched against a white reference spectrum. Optional Gaia DR3 TAP query (`vizier` feature flag) for real catalog data; built-in Bp-Rp estimation fallback.

3.17 Pixel Rejection Families (v0.6.0)

The stacking kernel (`core/stacking/combine.rs`) separates **rejection** from **combination**. For one output pixel let $\{(v_k, c_k)\}_{k=1}^N$ be the samples contributed by the N frames, where v_k is the stored value and c_k the comparison value used for the statistics (Section 3.18); without rejection normalization $c_k = v_k$. Non-finite samples are discarded before any test. Rejection selects a surviving subset; the combination step then acts on the v_k of the survivors only, never on the c_k .

All scale estimates share one fallback: the robust scale is $1.4826 \cdot \text{MAD}$, and when the MAD is zero it degrades to 1.2533141 times the mean absolute deviation about the same centre.

Sigma clipping. Iterative, with the keep test

$$\boxed{\text{keep } k \iff -\sigma_{\text{low}} s \leq c_k - \mu \leq \sigma_{\text{high}} s} \quad (21)$$

On the first iteration $\mu = \text{median}(c)$ and $s = 1.4826 \cdot \text{MAD}(c)$; on every later iteration μ is the mean and s the sample standard deviation (divisor $N - 1$). The loop stops at `max_iterations` (default 5), when an iteration removes nothing, when $s \leq 0$, or when fewer than two samples remain. Defaults are $\sigma_{\text{low}} = 4.0$ and $\sigma_{\text{high}} = 3.0$.

Winsorized sigma clipping. The centre stays the median at every iteration; only the scale changes. With $s_0 = 1.4826 \cdot \text{MAD}(c)$ and cutoff κ (default 5.0), the samples are first Winsorized,

$$\tilde{c}_k = \text{clamp}(c_k, \mu - \kappa s_0, \mu + \kappa s_0), \quad (22)$$

and a fixed-point iteration then re-estimates the scale with the tails held at ± 1.5 current sigmas:

$$u_k^{(j)} = \text{clamp}(\tilde{c}_k, \mu - 1.5 s^{(j)}, \mu + 1.5 s^{(j)}) \quad (23)$$

$$s^{(j+1)} = 1.134 \cdot \text{stdev}(u^{(j)}) \quad (24)$$

Tails are replaced, not discarded

Every sample outside the window is *set to the window edge* and stays in the sum; the divisor remains N (variance uses $N - 1$). The estimator therefore keeps the full sample count while the outliers lose their leverage, which is what distinguishes it from trimming. The factor 1.134 restores consistency with the Gaussian standard deviation after that replacement. At most 10 steps, stopping when the relative change falls below 10^{-4} .

The keep test is then Equation 21 with μ the median and s the Winsorized sigma. Needs at least three samples.

Linear fit clipping. The samples are sorted by c and a straight line in *rank* is fitted by least squares over the inner half of the ranks, that is $i \in [\lfloor n/4 \rfloor, n - \lfloor n/4 \rfloor]$:

$$c_{(i)} \approx a + b i, \quad r_i = c_{(i)} - (a + b i), \quad d = 1.4826 \cdot \text{median}_i |r_i| \quad (25)$$

Residuals are evaluated at *all* ranks, not only the fitted ones, and the keep test is $-\sigma_{\text{low}}d \leq r_i \leq \sigma_{\text{high}}d$. The method needs at least 5 samples; below that the kernel falls back to plain sigma clipping.

Percentile clipping. A single non-iterative pass about the median $m = \text{median}(c)$, with bounds expressed as fractions of the median magnitude:

$$\text{keep } k \iff -p_{\text{low}} |m| \leq c_k - m \leq p_{\text{high}} |m| \quad (26)$$

Defaults $p_{\text{low}} = 0.2$ and $p_{\text{high}} = 0.1$. Needs at least 3 samples and a finite, non-zero $|m|$; otherwise the set passes untouched.

Min/max clipping. No statistics at all: sort by c , drop the n_{low} smallest and the n_{high} largest (both default 1). Skipped entirely when $N \leq n_{\text{low}} + n_{\text{high}}$.

Method	Criterion	Iterative	Min. N
Sigma clip	deviation from median/mean in units of s	yes	2
Winsorized sigma clip	same, s from tail replacement	yes	3
Linear fit clip	residual from a line fitted in rank	yes	5
Percentile clip	deviation as a fraction of $ m $	no	3
Min/max	fixed count from each end	no	$n_{\text{low}} + n_{\text{high}} + 1$

Table 1: Rejection families and their guards (`RejectionParams`).

The combination step. Applied to the surviving v_k :

$$S = \begin{cases} \frac{1}{K} \sum_k v_k & \text{mean, unweighted} \\ \frac{\sum_k w_{f(k)} v_k}{\sum_k w_{f(k)}} & \text{mean, per-frame weights } w \\ \text{median}_k v_k, \quad \min_k v_k, \quad \max_k v_k & \text{median, min, max} \end{cases} \quad (27)$$

where $f(k)$ is the frame index of sample k . Weights act on the mean only; the weighted form reverts to the plain mean when $\sum w \leq 10^{-12}$. If every sample is rejected the pixel takes the last centre used by the rejection loop, or 0 when no finite centre was computed. With `rejection_maps` enabled the kernel also returns two `u16` maps holding the low and high rejection counts per pixel.

3.18 Frame Normalization (v0.6.0)

Normalization brings frames onto a common photometric footing before rejection. The per-frame statistics are computed on the **mutual overlap**: a linear pixel index contributes only when the value is finite in *every* frame, so all frames are described over exactly the same set of pixels. The index is strided so that at most 131 072 positions are sampled. For each frame the location is the median and the scale is $1.4826 \cdot \text{MAD}$ over that set; frame 0 is the reference (m_0, s_0).

Each frame f receives an affine map $v \mapsto k_f v + z_f$ with

$$(z_f, k_f) = \begin{cases} (0, 1) & \text{none} \\ (m_0 - m_f, 1) & \text{additive} \\ (0, m_0/m_f) & \text{multiplicative} \\ \left(m_0 - \frac{s_0}{s_f}m_f, \frac{s_0}{s_f}\right) & \text{additive with scaling (default)} \\ (0, s_0/s_f) & \text{multiplicative with scaling} \end{cases} \quad (28)$$

The plain forms equalise the sky level only; the scaling variants also equalise the dispersion, so that frames taken under different transparency or sky brightness contribute noise of comparable amplitude. Both ratios are guarded: s_0/s_f becomes 1 when either scale is non-positive or non-finite, and m_0/m_f becomes 1 when $|m_f| \leq 10^{-12}$.

Scale + offset rejection normalization is a separate, non-destructive stage. After the maps above have been applied, the post-normalization statistics are $m'_f = k_f m_f + z_f$ and $s'_f = k_f s_f$, and each frame gets a relative scale $\rho_f = s'_f/s'_0$. The rejection kernel then sees a comparison value

$$c_k = \frac{v_k - m'_f}{\rho_f} + m'_0 \quad (29)$$

while the value entering the combination remains v_k . Rejection decisions are therefore taken in the reference frame's units, but the stacked result is still built from the frames' own pixels. A frame with $m'_f = m'_0$ and $\rho_f = 1$ is left alone. Statistics are computed whenever either normalization or scale + offset rejection normalization is requested.

3.19 Thin-Plate Spline Background Extraction — DBE (v0.6.0)

The Spline background model (`core/imaging/dbe.rs`) fits a **regularised thin-plate spline** through box samples, in place of the global polynomial surface of the Background Extraction subsection. Coordinates are divided by $S = \max(\text{rows}, \text{cols})$ before the fit, so the node spacing is $O(1)$.

The radial basis function is the two-dimensional thin-plate kernel

$$\varphi(r) = r^2 \ln r \quad (r > 0), \quad \varphi(0) = 0 \quad (30)$$

and the model is that kernel plus an affine term,

$$B(x, y) = a_0 + a_1 x + a_2 y + \sum_{j=1}^n \lambda_j \varphi(\|\mathbf{p} - \mathbf{p}_j\|) \quad (31)$$

The coefficients come from one $(n+3) \times (n+3)$ system solved by Gaussian elimination with partial pivoting:

$$\begin{pmatrix} \mathbf{K} + \mu \mathbf{W}^{-1} & \mathbf{P} \\ \mathbf{P}^\top & \mathbf{0} \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{a} \end{pmatrix} = \begin{pmatrix} \mathbf{z} \\ \mathbf{0} \end{pmatrix}, \quad K_{ij} = \varphi(\|\mathbf{p}_i - \mathbf{p}_j\|), \quad \mathbf{P} = \begin{pmatrix} 1 & x_1 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & x_n & y_n \end{pmatrix} \quad (32)$$

The three zero rows are the usual side conditions $\sum_j \lambda_j = \sum_j \lambda_j x_j = \sum_j \lambda_j y_j = 0$, which make the affine part unique. Regularisation is the diagonal term

$$(\mu \mathbf{W}^{-1})_{ii} = \frac{\mu}{\max(w_i, 10^{-6})}, \quad \mu = \text{smoothing} \cdot \begin{cases} \bar{d}^2 |\ln \bar{d}| & 0 < \bar{d} < 1 \\ 1 & \text{otherwise} \end{cases} \quad (33)$$

where $\bar{d}$ is the mean nearest-neighbour distance between scaled nodes, so the penalty is scaled by the kernel's own magnitude at the typical node spacing. Smoothing defaults to 0.25. Because μ is divided by the sample weight w_i , a poorly filled sample box is smoothed harder than a fully covered one; at $\mu = 0$ the spline interpolates the samples exactly.

Sample placement. Manual samples (Point regions) are taken first. The automatic grid divides the image into $g \times g$ cells (g default 10, at least 2, each cell at least 4 px per axis), trims a margin of a quarter cell on each side and places the sample at the centre of the remaining inner box. Automatic positions within 1 px of a manual one are dropped, and the automatic set is thinned uniformly so that the total does not exceed `max_samples` (default 400).

Each sample is a box of half-width `sample_radius` (default 5, so 11×11 px) reduced by sigma-clipped statistics ($\kappa = 2.5$, 3 iterations) to a median, a sigma and a maximum; the sample value is the median and its weight is the finite fraction of the nominal box area, capped at 1.

Sample rejection has four recorded reasons:

- **bounds** — the rounded centre falls outside the image
- **finite** — fewer than 25% of the nominal box pixels are finite
- **star** — the box maximum exceeds $\text{median} + 5 \max(\sigma_{\text{sky}}, \sigma_{\text{box}})$, where σ_{sky} is the median of the usable per-box sigmas
- **deviation** — after a first fit, the residual exceeds $\max(\text{tolerance} \cdot \sigma_{\text{sky}}, 3\sigma_{\text{res}})$, with σ_{res} the MAD-based sigma of the residuals and tolerance default 1.0

Deviation rejection runs once and the spline is refitted on the survivors; it is skipped when it would leave fewer than 4 accepted samples, which is also the minimum for any fit. The final model is evaluated on a coarse lattice of step $\max(8, \min(\text{rows}, \text{cols})/256)$ and bilinearly interpolated to full resolution, then applied by subtraction or by division (guarded at $B > 10^{-10}$). The reported RMS residual is taken over the accepted samples only.

3.20 Aperture Photometry with Fractional Pixel Weights (v0.6.0)

`core/analysis/aperture.rs` replaces the centre-in-circle rule with a sub-pixel coverage weight. For a pixel whose centre is at distance d from the aperture centre, with $\delta = 1/\sqrt{2}$ the half-diagonal of a pixel:

$$w = \begin{cases} 1 & d + \delta \leq r \\ 0 & d - \delta > r \\ \frac{1}{n^2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{1}[\|\mathbf{s}_{ij} - \mathbf{c}\|^2 \leq r^2] & \text{otherwise} \end{cases} \quad (34)$$

where the sampling points are $\mathbf{s}_{ij} = \mathbf{p} + \left(\frac{i+0.5}{n} - \frac{1}{2}, \frac{j+0.5}{n} - \frac{1}{2}\right)$, an $n \times n$ grid over the pixel with $n = \text{subsamples}$, default 5. Only the boundary band is sampled; interior and exterior pixels take the closed forms. An annulus weight is the difference of the two circle fractions, $w_{\text{out}} - w_{\text{in}}$, evaluated on the same grid.

The aperture integral and its effective area are

$$F = \sum_p w_p I_p, \quad A = \sum_p w_p \quad (35)$$

over pixels that are finite and not set in the exclusion mask; masked and non-finite pixels are counted separately and reported. A is what `aperture_pixels` rounds, so it tracks πr^2 rather than the count of partially covered pixels.

Local background. The sky annulus has radii $2r_{\text{ap}}$ and $3r_{\text{ap}}$ about the refined centroid. Only annulus pixels with $w \geq 0.5$ are taken as members. From the finite values in that set, $\bar{b}$ is the mean of the interquartile range (ranks $\lfloor n/4 \rfloor$ to $\lfloor 3n/4 \rfloor$) and $\sigma_{\text{sky}} = 1.4826 \cdot \text{median}_i |b_i - \text{median}(b)|$ over the whole set. The net flux is $F_{\text{net}} = F - \bar{b}A$ and the instrumental magnitude $m = -2.5 \log_{10} F_{\text{net}}$.

The aperture radius defaults to $1.5 \times \text{FWHM}$ clamped to $[2, 60]$ px, with the FWHM taken from second moments truncated at half maximum and corrected by the factor $1 - \ln 2$ for that truncation.

Error propagation always carries the uncertainty of the sky level itself,

$$\sigma_b^2 = \frac{A^2 \sigma_{\text{sky}}^2}{n_{\text{bg}}}, \quad (36)$$

and then takes one of two forms depending on whether an ERR plane accompanies the data:

$$\sigma_F^2 = \sum_p (w_p e_p)^2 + \sigma_b^2 \quad \text{ERR plane present} \quad (37)$$

$$\sigma_F^2 = \frac{\max(F_{\text{net}}, 0)}{g} + A \sigma_{\text{sky}}^2 + \sigma_b^2 \quad \text{no ERR plane} \quad (38)$$

In Equation 37 each pixel's quoted error is scaled by the same fractional weight used for the flux. In Equation 38 the Poisson term is present only when a gain g in electrons per ADU is supplied, and is zero otherwise; the second term is the shot noise of the sky within the aperture. The result reports which of the two was used, and $\text{SNR} = F_{\text{net}}/\sigma_F$.

Curve-of-growth aperture correction. The cumulative net flux is measured at integer radii out to $4r_{\text{ap}}$,

$$C(r) = \sum_{p \in \mathcal{A}(r)} w_p I_p - \bar{b} \sum_{p \in \mathcal{A}(r)} w_p \quad (39)$$

and the plateau is the first radius $r \geq \max(\lfloor r_{\text{ap}} \rfloor + 1, 3)$ satisfying

$$C(r) > 0 \quad \text{and} \quad |C(r) - C(r-2)| < 0.01 C(r) \quad (40)$$

that is, the growth curve has flattened to better than 1% over two radial steps. If that radius reaches the inner edge of the sky annulus, $2r_{\text{ap}}$, no correction is reported, because the curve would then be contaminated by the region used to define the background. Otherwise the correction factor is $F_{\text{net}}/C(r_{\text{plateau}})$ and $C(r_{\text{plateau}})$ is reported as the total flux.

3.21 Barycentric and Heliocentric Radial Velocity (v0.6.0)

`core/astrometry/spectral.rs` computes the correction from the header alone, using the **Meeus low-precision ephemeris** (*Astronomical Algorithms*, ch. 25 for the Sun and a truncated ch. 47 for the Moon). Times come from `core/astrometry/time.rs`: Julian dates through the Meeus calendar algorithm, T in Julian centuries from J2000.0, and GMST from

$$\theta_{\text{GMST}} = 280.46061837 + 360.98564736629 d + 0.000387933 T^2 - \frac{T^3}{38\,710\,000} \quad (41)$$

with $d = \text{JD} - 2451545.0$. UTC is treated as UT1 and $\text{TT} = \text{UTC} + 69.2\text{ s}$.

Solar coordinates.

$$L_0 = 280.46646 + 36000.76983 T + 0.0003032 T^2 \quad (42)$$

$$M = 357.52911 + 35999.05029 T - 0.0001537 T^2 \quad (43)$$

$$e = 0.016708634 - 0.000042037 T - 0.0000001267 T^2 \quad (44)$$

$$C = (1.914602 - 0.004817 T - 0.000014 T^2) \sin M + (0.019993 - 0.000101 T) \sin 2M + 0.000289 \sin 3M \quad (45)$$

$$\odot = L_0 + C, \quad R = \frac{1.000001018 (1 - e^2)}{1 + e \cos(M + C)} \quad (46)$$

The Earth–Moon barycentre sits at heliocentric ecliptic longitude $\odot + 180^\circ$ and radius R ; the Earth itself is offset from it by the geocentric lunar vector divided by $1 + 81.30056$. The heliocentric velocity is a central difference of that position with $\Delta t = 1/48$ day, converted with $1 \text{ AU} = 149\,597\,870.7 \text{ km}$, then rotated from the ecliptic of date to equatorial J2000 by $R_z(-(1.3969713 T + 0.0003086 T^2))$ followed by $R_x(23.4392911^\circ)$.

Diurnal term. On the WGS84 ellipsoid ($a = 6\,378\,137 \text{ m}$, $f = 1/298.257223563$, $e^2 = 2f - f^2$), with $N = a/\sqrt{1 - e^2 \sin^2 \phi}$ the prime vertical radius, the site's distance from the rotation axis and its velocity are

$$\rho = (N + h) \cos \phi, \quad \mathbf{v}_{\text{rot}} = \omega \rho (-\sin \theta_{\text{LST}}, \cos \theta_{\text{LST}}, 0), \quad \omega = 7.292115 \times 10^{-5} \text{ rad s}^{-1} \quad (47)$$

with $\theta_{\text{LST}} = \theta_{\text{GMST}} + \lambda$ and longitude read east-positive.

Projection. With the target unit vector $\hat{\mathbf{u}} = (\cos \delta \cos \alpha, \cos \delta \sin \alpha, \sin \delta)$,

$$v_{\text{helio}} = \mathbf{v}_{\oplus} \cdot \hat{\mathbf{u}} + \mathbf{v}_{\text{rot}} \cdot \hat{\mathbf{u}} \quad (48)$$

$$v_{\text{bary}} = v_{\text{helio}} + \mathbf{v}_{\odot} \cdot \hat{\mathbf{u}} \quad (49)$$

where $\mathbf{v}_{\odot} = -\sum_p \mathbf{v}_p m_p / M_{\odot}$ is the Sun's reflex motion about the solar-system barycentre, modelled as circular orbits for Jupiter, Saturn, Uranus and Neptune. The sign convention is that the correction is *added* to a topocentric velocity, positive when the observer moves toward the target.

Quoted accuracy

The correction is reported with `accuracy_kms` = 0.02 km s⁻¹ and the method string *Meeus low-precision ephemeris*. A SPECSYS of GEOCENTR skips the diurnal term; frames already barycentric or beyond report a zero shift with the frame named; spacecraft headers (TELESCOP containing JWST, HST or ROMAN) take the correction as -VELOSYS/1000 in km s⁻¹ from the header, or are refused with the reason, since the spacecraft ephemeris is not reconstructible from the header.

Air and vacuum wavelengths. Following Greisen et al. (2006), FITS Paper III eq. 65, with λ in micrometres,

$$n(\lambda) = 1 + 10^{-6} \left(287.6155 + \frac{1.62887}{\lambda^2} + \frac{0.01360}{\lambda^4} \right) \quad (50)$$

The forward direction is direct, $\lambda_{\text{air}} = \lambda_{\text{vac}}/n(\lambda_{\text{vac}})$. The inverse has no closed form and is solved by fixed-point iteration $\lambda^{(k+1)} = \lambda_{\text{air}} n(\lambda^{(k)})$ started at $\lambda_{\text{air}} n(\lambda_{\text{air}})$, converging at 10^{-10} μm within at most 50 steps. Below 0.2 μm the formula is outside its stated range and the wavelength is passed through unchanged, with a note.

Velocity conventions. With $c = 299\,792.458$ km s⁻¹, observed λ and rest λ_0 :

$$v_{\text{opt}} = c \frac{\lambda - \lambda_0}{\lambda_0} \quad \lambda = \lambda_0(1 + \beta) \quad (51)$$

$$v_{\text{rad}} = c \frac{\lambda - \lambda_0}{\lambda} \quad \lambda = \frac{\lambda_0}{1 - \beta} \quad (52)$$

$$v_{\text{rel}} = c \frac{\lambda^2 - \lambda_0^2}{\lambda^2 + \lambda_0^2} \quad \lambda = \lambda_0 \sqrt{\frac{1 + \beta}{1 - \beta}} \quad (53)$$

with $\beta = v/c$; the right-hand column is the inversion used to place a velocity back on the wavelength axis. The three agree to first order in β and diverge beyond it.

3.22 Moment Maps (v0.6.0)

`core/cube/moments.rs` collapses a spectral cube over a channel range $[z_0, z_1]$ into three maps. The spectral axis is first converted to velocity by the convention of Section 3.21; an axis already stored in velocity units is used as-is. Channel widths come from a centred difference,

$$\Delta v_z = \begin{cases} |v_1 - v_0| & z = 0 \\ |v_{n-1} - v_{n-2}| & z = n - 1 \\ \frac{1}{2} |v_{z+1} - v_{z-1}| & \text{otherwise} \end{cases} \quad (54)$$

Per-pixel continuum. Two side windows $[z_a, z_b]$ and $[z_c, z_d]$ are merged, sorted and deduplicated. When the two windows differ and at least three channels remain, a first-order baseline is fitted per pixel by ordinary least squares in the channel index; otherwise the fit collapses to a constant, and it also collapses when fewer than three channels are finite or the normal equations are degenerate:

$$b_i(z) = \alpha_i + \beta_i z, \quad \beta_i = \frac{n \sum zy - \sum z \sum y}{n \sum z^2 - (\sum z)^2}, \quad \alpha_i = \frac{\sum y - \beta_i \sum z}{n} \quad (55)$$

The per-pixel noise σ_i is the MAD-based robust sigma of that fit's residuals, $1.4826 \cdot \text{MAD}$. With no windows the raw channel values are used and a single global sigma is taken from a line-free channel, $z_0 - 1$ if it exists, otherwise $z_1 + 1$.

The moments. With the continuum-subtracted sample $s_{i,z} = I_{i,z} - b_i(z)$:

$$M_0 = \sum_{z=z_0}^{z_1} s_{i,z} \Delta v_z \quad (56)$$

$$M_1 = \frac{\sum_{z:s>0} s_{i,z} v_z}{\sum_{z:s>0} s_{i,z}} \quad (57)$$

$$M_2 = \sqrt{\max\left(\frac{\sum_{z:s>0} s_{i,z} v_z^2}{\sum_{z:s>0} s_{i,z}} - M_1^2, 0\right)} \quad (58)$$

M_0 is the integrated intensity and takes every finite channel, positive or negative, weighted by its own channel width. M_1 and M_2 are the intensity-weighted mean velocity and its dispersion, and both restrict the sum to *positive* samples so that negative noise excursions cannot drive the weights or push the variance negative; the variance is clamped at zero before the root regardless.

SNR masking. The noise used for a pixel is the larger of its own residual sigma and a global floor, the map median of the per-pixel sigmas. A pixel passes when

$$\frac{M_0}{\sigma_i \sqrt{N_{\text{ch}}} \overline{\Delta v}} \geq t \quad (59)$$

with $N_{\text{ch}} = z_1 - z_0 + 1$, $\overline{\Delta v}$ the mean channel width over the range and t the threshold, default 3.0. The $\sqrt{N_{\text{ch}}} \overline{\Delta v}$ factor is the propagated noise of the M_0 sum itself. M_1 and M_2 are written only where the test passes; M_0 is kept for failing pixels when `mask_below_threshold` is disabled, and stays NaN otherwise. When no noise estimate is available the pixel is passed and counted as unmaskable.

3.23 Wavelet Per-Scale Detail Bias (v0.6.0)

`core/imaging/wavelet.rs` implements the à trous (“with holes”) transform with the B_3 spline kernel

$$h = \frac{1}{16} [1, 4, 6, 4, 1] \quad (60)$$

applied separably, with a hole spacing of 2^k taps at scale k , edge clamping, and renormalisation over the finite taps only so that NaN pixels do not spread. Starting from $c_0 = I$:

$$c_{k+1} = h_k * c_k, \quad w_{k+1} = c_k - c_{k+1}, \quad I = \sum_{k=1}^N w_k + c_N \quad (61)$$

Up to 8 scales, default 5. The decomposition is exact: the layers and the residual sum back to the input.

Per-scale threshold. The noise is estimated once from the finest layer, $\sigma_{w_1} = 1.4826 \cdot \text{median}|w_1|$, and referred back to the image by the B_3 scaling table

$$\sigma = \frac{\sigma_{w_1}}{\kappa_0}, \quad \kappa = (0.8908, 0.2007, 0.0856, 0.0413, 0.0205, 0.0103, 0.0051), \quad \kappa_k = \frac{\kappa_6}{2^{k-6}} \quad (k > 6) \quad (62)$$

so that scale k is thresholded at its own expected noise amplitude:

$$T_k = t_k \sigma \kappa_k \quad (63)$$

with per-scale multipliers t defaulting to (3, 2.5, 2, 1.5, 1); scales beyond the supplied list reuse the last entry. Thresholding is soft by default, $w' = \text{sgn}(w) \max(|w| - T_k, 0)$, or hard, $w' = w \cdot \mathbf{1}[|w| > T_k]$.

Detail bias. Reconstruction applies a per-layer gain rather than summing the layers back at unit weight:

$$I' = c_N + \sum_{k=1}^N (1 + \beta_k) w'_k \quad (64)$$

The bias β_k is exposed on $[-1, +3]$ in steps of 0.05, default 0, and layers with no entry take $\beta_k = 0$. At $\beta_k = 0$ the layer is restored as it stands, so a run with all biases zero and all thresholds zero is the identity. At $\beta_k = -1$ the gain vanishes and the scale is removed from the image entirely; positive values amplify structure at that scale, up to four times its measured amplitude at $\beta_k = 3$. Because each β_k multiplies one layer of an exact decomposition, the operation is a linear per-scale filter and does not move energy between scales.

3.24 Local Histogram Equalization and HDR Multiscale Transform (v0.6.0)

LHE (`core/imaging/local_contrast.rs`) is contrast-limited adaptive histogram equalization. The image is covered by a grid of tiles of side $R = \text{kernel_radius}$, clamped to $[16, 512]$ with default 64, and a lookup table is built at each tile centre from the pixels within R of it — a disc when `circular` is set, the bounding square otherwise. Values are mapped onto $B = 2^b$ bins, $b \in \{8, 10, 12\}$ with default 8, by

$$p(v) = \text{clamp}\left(\frac{v - v_{\min}}{v_{\max} - v_{\min}}, 0, 1\right) (B - 1), \quad j = \lfloor p(v) + \tfrac{1}{2} \rfloor \quad (65)$$

over the finite range of the whole image.

Clip limit and redistribution. With n pixels in the window and contrast limit γ clamped to $[1, 64]$ (default 2.0),

$$c = \max\left(\frac{\gamma n}{B}, 1\right), \quad E = \sum_{j=0}^{B-1} \max(h_j - c, 0), \quad p_j = \min(h_j, c) + \frac{E}{B} \quad (66)$$

The clip limit is therefore a multiple of the *mean* bin population: $\gamma = 1$ flattens the histogram to uniform and gives no local contrast gain, while larger values allow progressively steeper transfer curves. The clipped mass E is not discarded but spread uniformly over all B bins, so $\sum_j p_j = n$ exactly and the transfer function remains a valid CDF. The lookup table is the mid-point corrected cumulative sum

$$L_j = \text{clamp}\left(\frac{1}{n} \left(\sum_{i \leq j} p_i - \frac{1}{2} p_j \right), 0, 1\right) \quad (67)$$

Each output pixel is interpolated linearly between the two adjacent bins of a table and bilinearly between the four surrounding tile tables, then blended with the input by the amount $a \in [0, 1]$ and clamped back into the input range:

$$v' = \text{clamp}((1 - a)v + a\hat{v}, v_{\min}, v_{\max}) \quad (68)$$

Non-finite pixels are passed through, and an image whose finite range is below 10^{-12} is returned unchanged.

HDRMT (`core/imaging/hdr.rs`) compresses large-scale dynamic range through the same à trous decomposition. Each of `iterations` passes (1–4, default 1) does:

1. decompose the working image into N detail layers (2–8, default 6) plus the residual c_N ;
2. clamp the residual from above at its own median, $c_N \leftarrow \min(c_N, \text{median}(c_N))$;
3. reconstruct with the per-layer bias of Section 3.23, $\beta_k = -\text{overdrive}$ for the two coarsest detail layers ($k + 2 \geq N$) and $\beta_k = 0$ elsewhere, with overdrive clamped to $[0, 1]$;
4. rescale the result to $[0, 1]$.

Step 2 is the transform itself: every pixel whose large-scale background sits above the median is pulled down to it, so a bright core and a faint outskirts end up on the same footing, while the detail layers that carry the structure are reinjected untouched. Optional inversion runs the whole pass on $1 - v$ to compress dark rather than bright structure; optional deringing blends the original normalised image back under a star mask with weight $m \cdot a$. The image is finally mapped back to its original range. On RGB the transform runs on the BT.709 luminance ($0.2126R + 0.7152G + 0.0722B$) and the ratio is applied to the three channels, so hue is preserved.

Why linear input is refused

HDRMT rejects any image whose finite median is below 1% of its finite maximum, with the message “*HDRMT expects a non-linear (stretched) image; median is below 1% of the maximum*”. On linear astronomical data almost every pixel sits just above the sky floor while the stars span several decades, so the residual median of step 2 would lie at that floor and the clamp would flatten essentially the whole frame. The transform assumes a stretch has already redistributed the histogram, which is why it is placed after the STF in the pipeline rather than before it.

4 Command Module Reference

The backend registers **107 Tauri commands**, all of them listed in the `tauri::generate_handler!` macro in `src-tauri/src/lib.rs`; the table below groups them by their `cmd::` module path.

The command name strings appear nowhere in the React components. They live only in the service layer, `src/services/*.ts`, where each exported function passes the literal name to

`typedInvoke` (`src/infrastructure/tauri/client.ts`). Arguments cross the boundary in camel-Case and arrive in Rust as snake_case — `computeStatistics` sends `excludeDq` and `compute_statistics_cmd` receives `exclude_dq` — so a rename on either side is a runtime failure, not a compile error.

Module	Commands	Area
<code>cmd::io</code>	4	FITS/ASDF ingestion, linear mono and RGB preview buffers
<code>cmd::export</code>	5	FITS and PNG export, mono and RGB, RICE-compressed MEF
<code>cmd::metadata</code>	5	Headers, HDU and ASDF array listings, narrowband filter detection
<code>cmd::analysis</code>	6	Histograms, FFT power spectrum, star detection, photometry, subframe metrics
<code>cmd::regions</code>	5	Region statistics, radial profiles, line cuts, DS9 <code>.reg</code> import and export
<code>cmd::visualization</code>	8	STF render, scale limits, colormap LUT, pixel probe, DQ flag table and mask preview, tile pyramids
<code>cmd::stacking</code>	5	Calibration, integration, drizzle (mono and RGB), batch pipeline
<code>cmd::compose</code>	11	Channel blend, alignment, crop, LRGB, white balance, SCNR, restretch, composite cache
<code>cmd::processing</code>	21	Resample, debayer, deconvolution, background and DBE, wavelets, arcsinh/GHS/masked stretch, star removal, tone, LHE, HDRMT, mono and composite variants
<code>cmd::pixelmath</code>	2	Expression evaluation and expression validation
<code>cmd::cosmetic</code>	2	Cosmetic correction, single frame and batch
<code>cmd::statistics</code>	3	Exact statistics (mono and composite) and batch noise evaluation
<code>cmd::cube</code>	8	Cube processing, frames, spectra, region spectra, range collapse, moment maps
<code>cmd::spectral</code>	2	Spectral axis construction and radial-velocity correction
<code>cmd::cutout</code>	1	Science cutout export (SCI, ERR, DQ)
<code>cmd::catalog</code>	3	Gaia DR3 cone search, cross-match, CSV export
<code>cmd::astrometry</code>	5	Plate solving, WCS info, pixel-to-world, pointing overlap, grid lines
<code>cmd::psf</code>	1	PSF estimation
<code>cmd::spcc</code>	1	Spectrophotometric colour calibration
<code>cmd::config</code>	4	Configuration read and write, API key storage
<code>cmd::synth</code>	2	Synthetic frame and synthetic stack generation
<code>cmd::output</code>	3	Output directory info, cleanup, progress cancellation
Total	107	

Table 2: The 107 commands registered in `generate_handler!`, grouped by `cmd::` module. Counts read off the macro block in `src-tauri/src/lib.rs`.

5 FITS and ASDF I/O

FITS via `memmap2` with BSCALE/BZERO: $v_{\text{phys}} = \text{BSCALE} \times v_{\text{stored}} + \text{BZERO}$. Multi-BITPIX export: 16 (auto BZERO/BSCALE), -32 (float32), -64 (float64).

ASDF: first Rust implementation. YAML tree $\rightarrow$ `NdArrayMeta` $\rightarrow$ binary block (zlib/bzip2/lz4) $\rightarrow$ `Array2`. Roman data model traversal. Auto-dispatch by extension. The reader was rewritten in 0.5.8 around `memmap2` with lazily decoded blocks and validated against the ASDF Standard.

5.1 Image References (v0.6.0)

Every path-based command accepts a reference rather than a bare path: `<path>#hdu=<n>` selects any FITS HDU and `<path>#array=<key>` any ASDF array, including dotted keys such as `roman.dq`. Parsing lives in `types/image_ref.rs`; a single loader in `infra/image_source.rs` resolves references for both the desktop commands and the headless server. The reference, not the path, is the cache key, and output filenames derive their stem from it, so `jw01234_cal.fits#hdu=3` writes `jw01234_cal_hdu3.png`. This is what makes ERR, DQ and variance planes first-class images that can be displayed and measured like any other.

5.2 Lossless Integer Planes (v0.6.0)

A DQ plane is a bit field, so rounding it through `f32` destroys individual flags. BITPIX 8/16/32 HDUs and `int8–int32`, `uint8–uint32` and `bool8` ASDF arrays are therefore decoded a second time into an `IntPlane` that sits beside the float view in the cache and counts against the same budget. The BZERO offsets 32768 and 2147483648 that FITS uses to store unsigned data in signed types are applied in integer arithmetic, never through a float intermediate. Display and processing stay in `f32/f64`; only flag decoding reads the integer plane.

5.3 Multi-Extension Output (v0.6.0)

`write_mef_images` writes plain multi-HDU IMAGE files from `f32`, `i32` and `u32` arrays. The science cutout path uses it to emit SCI, ERR and DQ in one file with CRPIX shifted and LTV1/LTV2/LTM1_1/LTM2_2 recorded, so the cutout remains registerable against its parent. `export_fits` can additionally write RICE_1-compressed tiles with a quantization level, for BITPIX 16 or `–32` only.

6 WebGPU Compute Pipeline

WGSL fragment shader implementing MTF with symmetric denominator guard. `r32float` texture, 6-vertex fullscreen quad. Uniform buffer: `[dataMin, dataMax, shadow, midtone, highlight, W, H, pad]` (32 bytes). CPU worker fallback with identical `stfTransfer` function.

6.1 Display Path Unification (v0.6.0)

The shader is no longer MTF-only. It evaluates the six scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`) and samples a 256×1 LUT texture for the colormap, so a colormap change uploads 1 KiB rather than re-rendering on the CPU. Limits come from the four limit modes (`minmax`, `zscale`, `percentile`, `user`) computed in Rust by `compute_scale_limits_cmd`; the LUT itself comes from `get_colormap_lut_cmd`.

The point of the exercise is that the stretch, the limit algorithms and the float-to-byte rule are written once, in `core/imaging/scale.rs`, and three consumers agree on them: the WGSL shader, the TypeScript worker fallback, and the headless server which returns a PNG. The parity is asserted rather than assumed — a vitest case sweeps every scale kind and checks the WGSL port against the canonical transfer within `f32` rounding.

Device loss is recoverable. A failed initialisation used to be cached for the session, so one transient adapter error left the app on the CPU path until restart; the retry path now re-probes.

GPU scope

The GPU is used for display only. Stacking, alignment, drizzle, deconvolution, background extraction, PixelMath, wavelets and the compose pipeline all run on the CPU in Rust, parallelised with **rayon**. No processing result depends on a GPU being present.

7 Headless HTTP Server

AstroBurst ships a second binary that exposes the same imaging core over HTTP, with no window, no WebView and no Tauri runtime. It is an **axum** 0.7 service on **tokio**, built around short-lived sessions that hold a per-session image cache, so a script or notebook can open a FITS or ASDF file once and then issue many inspection calls against the decoded array instead of re-reading the file each time. The headless server was contributed by Jae-Joon Lee (<https://github.com/leejjoon>); the Cargo manifest records this on the server dependency block and on the `[[bin]]` entry, and every source file under `src-tauri/src/server/` carries the same note.

The server is gated behind the `server` Cargo feature, which is *not* in default. The feature pulls in `axum`, `tower`, `tower-http`, `dashmap`, `uuid`, `tokio-stream`, `tokio-util` and `env_logger`, all declared optional. The binary is `astroburst-server` (`src/server/main.rs`) and declares `required-features = ["server"]`, so it is only built when that feature is on. Because the desktop feature `tauri` is a default, the server build turns the defaults off and re-enables the non-UI ones explicitly:

Build

```
cargo build --no-default-features --features server,astrometry-net,asdf-full,vizier
```

On start the process reads its configuration from the environment, initialises `env_logger`, spawns the TTL cleaner and serves on the configured address. Every response carries an `x-request-id` header, echoed from the request when present and otherwise a fresh UUIDv4, applied as a `tower` middleware layer over the whole router.

Variable	Meaning	Default
<code>ASTROBURST_BIND</code>	Listen address	<code>127.0.0.1:8080</code>
<code>ASTROBURST_SESSION_TTL</code>	Idle seconds before a session is evicted	<code>900</code>
<code>ASTROBURST_SESSION_MAX</code>	Concurrent sessions	<code>8</code>
<code>ASTROBURST_JOBS_MAX</code>	Concurrent long-running jobs	<code>4</code>
<code>ASTROBURST_CACHE_MAX_ENTRIES</code>	Per-session cache entries	<code>32</code>
<code>ASTROBURST_CACHE_MAX_BYTES</code>	Per-session cache bytes	<code>2GiB</code>
<code>ASTROBURST_CLEANUP_INTERVAL</code>	Seconds between TTL sweeps	<code>60</code>
<code>ASTROBURST_LOG_LEVEL</code>	Default <code>RUST_LOG</code> value	<code>info</code>

Table 3: Server configuration (`server/config.rs`). An unparseable value is reported on stderr and replaced by the default rather than aborting startup.

7.1 Session Model

A session is created by `POST /v2/sessions` (or the identical `POST /sessions`), which answers 201 with a UUIDv4 `session_id`. Sessions live in a `dashmap` keyed by that id; there is no authentication and no per-session credential.

Each session owns four things (`server/session.rs`):

- an `ImageCache` — the same LRU cache the desktop app uses — bounded by both `cache_max_entries` and `cache_max_bytes`;
- a map of jobs, keyed by job id;
- the v2 state: a monotonic counter, an `active_ref` under an `RwLock`, and a map of `ImageMeta` records;
- a `last_accessed` timestamp.

Images are referenced by name inside the session rather than by path. The v2 routes mint a ref on each producing call — `img_0`, `img_1`, `cutout_0`, `bin_0` and so on, from a per-session counter — unless the caller supplies `name`. Each ref carries an `ImageMeta` record holding `image_ref`, `source`, `hdu`, `array`, `plane_ref`, `is_dq`, `width`, `height`, `wcs_present` and `extname`. The most recently opened, cut or switched image becomes the session's `active_ref`, and every consuming endpoint treats a missing `ref` field as “use the active ref”, failing with 400 if there is none. The older slot-based routes instead take a caller-chosen `slot` string, defaulting to the file path.

Lifetime is bounded on three axes. Session creation fails with 503 once `session_max` sessions exist. Any request that resolves a session touches its timestamp through the `SessionExtractor`, and `POST /v2/sessions/:sid/keepalive` exists to do only that. A background task ticks every `cleanup_interval`, skips any session with a running job, and removes sessions idle longer than `session_ttl`, logging the eviction. `DELETE /v2/sessions/:sid` removes one immediately.

Eviction inside a session is independent of session lifetime: the image cache drops least-recently-used entries when either bound is exceeded, which can silently invalidate a ref the client still holds. The server reconciles rather than pretending otherwise — `prune_evicted_meta` drops metadata whose cache entry is gone, and `reconcile_active_ref` additionally clears `active_ref` when it points at an evicted image. `GET /v2/sessions/:sid` and `GET /v2/sessions/:sid/images` run that reconciliation before reporting, so a listing never advertises an image the cache no longer holds.

7.2 The v2 API Surface

Method	Path	Purpose
<i>Liveness</i>		
GET	/health	Status, version, active and cumulative session counts
<i>Session lifecycle</i>		
POST	/v2/sessions	Create a session, return its id
GET	/v2/sessions/:sid	Active ref, image count, cache bytes
DELETE	/v2/sessions/:sid	Drop the session and its cache
POST	/v2/sessions/:sid/keepalive	Refresh the idle timer
<i>Images and structure</i>		
POST	/v2/sessions/:sid/open	Open a FITS or ASDF plane, become the active ref
POST	/v2/sessions/:sid/hdu	Re-open the active image at another HDU or array
GET	/v2/sessions/:sid/images	List refs with their metadata
GET	/v2/sessions/:sid/structure	HDU listing of the source file (FITS only)
GET	/v2/sessions/:sid/header	Header cards, optionally glob-filtered by keys
GET	/v2/sessions/:sid/wcs	WCS summary: projection, CRPIX/CRVAL, CD, scale, rotation, parity, SIP
<i>Astrometry</i>		
POST	/v2/sessions/:sid/wcs/pix2sky	Batch pixel $\rightarrow$ sky, in a named frame
POST	/v2/sessions/:sid/wcs/sky2pix	Batch sky $\rightarrow$ pixel
POST	/v2/sessions/:sid/wcs/separation	Angular separation between two sky or two pixel points
POST	/v2/sessions/:sid/wcs/grid	Coordinate grid lines and labels at a given density
<i>Measurement</i>		
POST	/v2/sessions/:sid/pixel	Probe a pixel with an odd $n \times n$ box, plus DQ and ERR companions
POST	/v2/sessions/:sid/stats	Region statistics: exact moments, sigma clipping, percentiles, noise
POST	/v2/sessions/:sid/histogram	Binned histogram over a region, auto or explicit range
<i>Derived images and display</i>		
POST	/v2/sessions/:sid/cutout	Cut a region into a new ref, optionally shifting the WCS
POST	/v2/sessions/:sid/bin	Block-mean downsample into a new ref
POST	/v2/sessions/:sid/render	Render a region to PNG with limits, stretch, colormap, overlays

Table 4: Every route registered in `server/router.rs` under /v2, plus /health. Paths are literal; :sid is the session id.

The v2 endpoints share one region grammar (`server/v2/region.rs`): a request may pass a pixel rectangle, a sky rectangle in arcminutes around an RA/Dec, or a full shape — the core `RegionShape` set — in pixel or sky coordinates. A region that leaves the frame is rejected with `region_out_of_bounds` unless the caller passes `clip: true`, and a sky region on an image without WCS is rejected with `wcs_required`.

`POST /v2/sessions/:sid/render` is the only route that does not answer JSON: it returns `image/png` and puts the resolved parameters — the region actually used, `vmin/vmax`, the limit algorithm, stretch, colormap, the binning factor applied to honour `max_dim`, the PNG’s arcsec-per-pixel

scale and the fraction of pixels clipped at each end — into an `x-render-resolved` JSON header. Crosshair and scalebar overlays are drawn into the buffer before encoding.

Alongside these, the router keeps an earlier slot-based surface: `POST /sessions` and, under `/sessions/:sid`, `fits/open` and `fits/header` for I/O; `image/render`, `image/stf` and `image/viewport` for STF-based PNG output; `stacking/stack`, `stacking/drizzle` and `pipeline/run` for batch work; and the three job routes below. These address images by slot and do not participate in the v2 ref or active-ref model.

7.3 Job Model

Only three routes start work that can outlive a request: `stacking/stack`, `stacking/drizzle` and `pipeline/run`. All three follow the same contract (`server/job.rs`, `server/handlers/`):

- **Admission.** The handler validates its parameters, then takes a permit from a Semaphore sized at `jobs_max` with `try_acquire_owned`. No permit means an immediate 429; the queue never builds up.
- **Submission.** A Job is created with a bounded 256-slot event channel, registered in `session.jobs`, and the work is moved onto `spawn_blocking` holding the permit. The handler answers 202 Accepted with `job_id`, `status`: "running" and the cache slot(s) the result will be stored under. The pipeline route additionally returns any DQ cosmetic warnings raised while inspecting the input frames.
- **Polling.** `GET /sessions/:sid/jobs/:jid` returns `id`, `action`, `status` (`running`, `done`, `error`, `cancelled`), `pct`, and `started_at/completed_at` as milliseconds since the epoch.
- **Streaming.** `GET /sessions/:sid/jobs/:jid/stream` is a Server-Sent Events stream of `progress`, `complete` and `error` events, with a 30-second keep-alive. The receiver is taken out of the job on first subscribe, so a second subscriber gets 409.
- **Cancellation.** `DELETE /sessions/:sid/jobs/:jid` cancels a running job's `CancellationToken`, marks it cancelled and returns the job record. Transitions out of `running` go through a compare-and-exchange, and `set_cancelled` is a plain store, so a worker that finishes after the cancel cannot overwrite the cancelled state with `done` or `error`.

Results are written into the session's image cache under the requested slot names; there is no separate result store and no download route for them. Sessions with a running job are skipped by the TTL cleaner, so a long stack cannot have its own session evicted underneath it.

7.4 What Is Shared, and What Is Not

The server is a second front end over the library crate, not a reimplement. It links `astroburst_lib` and calls the same core modules the Tauri commands call:

- `core/imaging/scale.rs` — `LimitMode`, `StretchKind`, `resolve_limits`, `normalize_and_stretch`: used by `server/v2/render/handler.rs` and by `cmd/visualization/mod.rs`, so limits and stretches resolve identically on both;
- `core/imaging/region.rs` — the shape set, `shape_to_pixel`, masked values: used by `server/v2/region.rs` and by `cmd/regions.rs`, `cmd/statistics.rs`, `cmd/cutout.rs`;
- `core/imaging/cutout.rs` — `resolve_cutout_rect`, `cut_plane_within`, `shift_header`, `reported_ltv`: used by `server/v2/cutout.rs` and `cmd/cutout.rs`;
- `core/imaging/pixel_probe.rs` — `probe_pixel`, `probe_companions`, `data_unit`: used by `server/v2/pixel.rs` and by `cmd/statistics.rs` and `cmd/visualization/mod.rs`.

The same holds for `core/imaging/colormap.rs`, `core/imaging/statistics.rs`, `core/astrometry/` (`wcs.rs`, `grid.rs`, `frames.rs`) and the infrastructure layer — `infra/cache.rs` and `infra/image_source.rs`

— all of which both binaries import.

The split is in the command layer, not the core. Radial profiles, line cuts and DS9 `.reg` import/export exist as core functions (`radial_profile` and `line_cut` in `core/imaging/region.rs`, and `core/imaging/region_file.rs`) but are reachable only through `cmd/regions.rs`; no route in `server/router.rs` exposes them. Likewise, the `exclude_dq` flag that drives `resolve_dq_mask` appears only under `cmd/` — in `cmd/analysis/mod.rs`, `cmd/regions.rs` and `cmd/statistics.rs` — and is absent from the server’s `StatsParams` and `HistogramParams`. Both README claims therefore hold as written.

DQ on the server

The asymmetry is narrower than “no DQ”. `POST /v2/sessions/:sid/pixel` does load the DQ and ERR companions of the probed plane and returns decoded DQ flags for the sampled pixel. What the server lacks is *DQ-masked* aggregation: its stats and histogram endpoints have no exclusion flag, so they measure every finite pixel in the region regardless of its DQ state.

Further desktop-only surface: the compose wizard has no server routes at all, and `histogram` rejects `render_png: true` with a `not_implemented` code rather than plotting server-side.

7.5 Error Model

All failures go through one enum in `server/error.rs` and serialise to a single envelope, `{"success": false, "error": {"code", "message", "hint"?}}`, where `hint` is present only on the variant that carries one.

Status	Code	Raised for
400	<code>bad_request</code>	Malformed path or body, unknown colormap, stretch, algorithm or frame, both <code>hdu</code> and <code>array</code> given, zero or oversized bin factor, no active image in the session
400	<code>pixel_out_of_bounds</code>	Pixel probe outside the array; hint gives the valid extent
400	<code>region_out_of_bounds</code>	Region off-image, non-overlapping, clipped without <code>clip: true</code> , or a cutout exceeding the session memory budget
400	<code>wcs_required</code>	Sky coordinates requested on an image with no header, no usable WCS, or a degenerate pixel scale
400	<code>not_implemented</code>	<code>render_png</code> on the histogram endpoint
404	<code>not_found</code>	Unknown session, image ref, slot or job id
409	<code>conflict</code>	Second SSE subscriber on one job
429	<code>too_many_requests</code>	No job permit available
503	<code>service_unavailable</code>	Session cap reached
500	<code>internal_error</code>	Any <i>anyhow</i> error reaching the boundary, including a panicking blocking task

Table 5: Status codes produced by `AppError`. The four typed 400 codes let a client distinguish a bad request from an out-of-frame one without parsing prose.

Two details are worth noting for clients. The 429 message is the fixed string `"job queue full (max 4 concurrent)"`, which states the default rather than the configured `jobs_max`; the status code, not the text, is the contract. And errors that originate in the core — `RegionError`, `CutoutError`, `ProbeError` — are mapped explicitly to 400 with a hint, so a caller mistake is never reported as 500. The exception is plane loading, where only the two recognised messages (no ASDF arrays, no HDU index) become 400 and everything else is surfaced as `internal_error`.

8 Performance Benchmarks

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096	120 ms	533 MB/s
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon)
Histogram + auto-STF	4096×4096	35 ms	NaN-safe
Star detection ($\sigma=5$)	4096×4096	80 ms	~ 3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iters
Drizzle $2\times$	10×64 MB	4.2 s	Flat accumulator
Channel blend (3ch)	$4096 \times 4096 \times 3$	0.4 s	+ auto STF + lum FITS
Affine alignment	4096×4096	0.8 s	80 stars, RANSAC
Masked stretch (10 iter)	4096×4096	1.2 s	Star mask + MTF
WebGPU STF render	4096×4096	8 ms	GPU compute
WB (non-destructive)	$4096 \times 4096 \times 3$	45 ms	ORIG read + factor

Table 6: Benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe). Binary: ~ 15 MB. Measured on the v0.4.6 build and *not* re-measured for 0.6.0: the kernels behind these rows are unchanged, but the figures should be treated as the order of magnitude rather than as current numbers. The operations added since — pixel rejection, DBE, PixelMath, LHE, HDRMT, moment maps — are not represented here at all.

9 Dependencies

The default feature set is `tauri`, `astrometry-net`, `asdf-full` and `vizier`; the headless server is built instead with `-no-default-features -features server,astrometry-net,asdf-full,vizier`.

Crate	Version	Purpose
tokio	1	Async runtime; every command is moved off the UI thread with <code>spawn_blocking</code>
serde	1	Derived serialization for the IPC and server payloads
serde_json	1	JSON command responses
serde_yaml	0.9	ASDF YAML tree parsing (<code>infra/asdf/parser.rs</code> , <code>tree.rs</code>)
anyhow	1.0	Error propagation inside commands
thiserror	2.0	Typed errors for the region, cutout and pixel-probe cores
ndarray	0.16	N-dimensional arrays, with the <code>rayon</code> and <code>serde</code> features
rayon	1.10	Data-parallel batch processing
rand	0.8	Seeded RNG for the synthetic data generator (<code>core/synth</code>) and tests
image	0.25	PNG encoding, 8 and 16 bit (<code>cmd/helpers.rs</code>)
rustfft	6.2	FFT for deconvolution, phase correlation and the power spectrum
num-complex	0.4	Complex arithmetic alongside <code>rustfft</code>
memmap2	0.9	Memory-mapped FITS and ASDF reads, lazy cube frames
flate2	1.0	zlib for ASDF blocks and GZIP_1/GZIP_2 FITS tiles
wcs	0.4	wcs-rs projection engine (~20 FITS projections); SIP is applied by AstroBurst's own math
zip	2	Transparent <code>.zip</code> FITS containers (<code>infra/fits/dispatcher.rs</code>)
tempfile	3.8	Scratch files for the plate-solve upload and for tests
dirs	5.0	Platform data and config directories
regex	1.10	Header keyword discovery (<code>core/metadata/header_discovery.rs</code>)
base64	0.22.1	Base64 previews in the batch pipeline response
log	0.4.29	Logging facade used by the commands and the server
libc	0.2	Linux target only: <code>fstats</code> identifies a network filesystem before a file is memory-mapped
tauri-build	2.5	Build dependency: Tauri context and asset generation

Table 7: Unconditional dependencies, from `src-tauri/Cargo.toml` at v0.6.0.

Crate	Version	Purpose (gating feature)
tauri	2.10	Application framework, IPC, asset protocol — <code>tauri</code>
tauri-plugin-dialog	2.6.0	Native file dialogs — <code>tauri</code>
tauri-plugin-fs	2.2.0	Filesystem access from the webview — <code>tauri</code>
tauri-plugin-shell	2.3	External process and shell access — <code>tauri</code>
tauri-plugin-opener	2.3	Open files and URLs in the system handler — <code>tauri</code>
axum	0.7	HTTP router of the headless server — <code>server</code>
tower	0.4	Service utilities (<code>ServiceExt</code> in the server test harness) — <code>server</code>
tower-http	0.5	Declared with the <code>request-id</code> , <code>trace</code> and <code>util</code> features — <code>server</code>
dashmap	6	Concurrent session and image maps — <code>server</code>
uuid	1	v4 session and job identifiers — <code>server</code>
tokio-stream	0.1	SSE job-event stream — <code>server</code>
tokio-util	0.7	Cancellation tokens for jobs — <code>server</code>
env_logger	0.11	Logger initialisation for the server binary — <code>server</code>
request	0.12	Astrometry.net upload and VizieR TAP — <code>astrometry-net</code> ; <code>vizier</code> adds <code>request/blocking</code>
bzip2	0.5	bzip2 ASDF blocks — <code>asdf-full</code>
lz4_flex	0.11	lz4 ASDF blocks — <code>asdf-full</code>

Table 8: Optional dependencies and the Cargo feature that gates each.

10 Known Limitations

1. **Plate solving is online only.** Solving uploads the frame to nova.astrometry.net and needs a free API key; there is no local index. This is the largest functional gap against comparable tools.
2. **No optical distortion correction of the image.** WCS SIP distortion *is* applied, by the wrapper in `core/astrometry/wcs.rs` rather than by the `wcs` crate, so coordinates are correct on SIP headers; what is absent is resampling the pixels to remove barrel or pincushion distortion.
3. **ASDF is read-only.** The reader is complete (memory-mapped, lazy blocks, validated against the ASDF Standard); no writer is implemented, so Roman products are exported as FITS.
4. **No MAST API integration.** Observatory data must be downloaded manually before it is opened.
5. **No session persistence.** Panel sizes and per-file regions survive a restart through `localStorage`; the loaded set of files, the processing chain and the composite state do not. There is no project file.
6. **WebGPU is preferred, not required.** The display path needs Chromium 113+; when WebGPU is unavailable or the device is lost the app falls back to a CPU worker that reproduces the same bytes.
7. **Server parity is partial.** Rendering, region statistics, histograms and cutouts are available over HTTP. Radial profiles, line cuts and DS9 `.reg` import/export are desktop-only, and the server statistics and histogram endpoints do not yet accept the DQ exclusion flag that the desktop commands take.
8. **Flatpak/Flathub distribution** is pending reviewer approval.

11 Roadmap

Feature	Priority	Status
Batch calibration and stacking pipeline	Medium	Done (v0.3.0)
Star-based affine alignment	High	Done (v0.4.0)
ComposeWizard pipeline	High	Done (v0.4.2)
Masked stretch	High	Done (v0.4.2)
SPCC calibration	High	Done (v0.4.2)
Non-destructive WB/SCNR	High	Done (v0.4.5)
STF GPU/CPU consistency	High	Done (v0.4.5)
Tone curves (AdjustStep)	High	Done (v0.4.6)
Cache-only processing	High	Done (v0.4.6)
Auto-STF preview	Medium	Done (v0.4.6)
Headless HTTP server and Python client	Medium	Done (v0.5.5.2)
Star removal (starless plus stars layer)	Medium	Done (v0.5.6.1)
Display controls: scale modes, limit modes, colormaps	High	Done (v0.5.8.1)
Multi-extension FITS: any HDU, any ASDF array, ERR/DQ	High	Done (v0.5.8.1)
DQ flag decoding, tables and overlays	High	Done (v0.5.8.1)
Interactive regions with DS9 .reg exchange	High	Done (v0.5.8.2)
Pixel rejection family and frame normalization	High	Done (v0.5.8.3)
DBE spline background	High	Done (v0.5.8.3)
PixelMath	High	Done (v0.5.8.3)
LHE and HDR multiscale transform	Medium	Done (v0.5.8.3)
Exact statistics with k-sigma noise evaluation	High	Done (v0.5.8.3)
Photometric calibration and Gaia DR3 catalog panel	Medium	Done (v0.5.8.4)
Spectral axes and velocity conventions	Medium	Done (v0.5.8.4)
Cube moment maps and range collapse	Medium	Done (v0.5.8.4)
Science cutout export (SCI/ERR/DQ)	Medium	Done (v0.5.8.4)
WCS coordinate grid overlay	Medium	Done (v0.5.8.4)
GPU viewer follows the processed result	High	Done (v0.6.0)
Flatpak/Flathub	Medium	Pending
Local plate-solving index (offline solver)	High	Planned
Masks as a first-class object	High	Planned
Process history, undo and presets	High	Planned
Local normalization	High	Planned
Mosaic assembly with overlap normalization	Medium	Planned
Comet and asteroid alignment	Medium	Planned
RAW reader (LibRaw or <code>rawloader</code>)	Medium	Planned
XISF reader	Medium	Planned
TIFF read and write (Astro-TIFF)	Medium	Planned
MAST query and download	Medium	Planned
Full GPU compute pipeline	Low	Planned (v1.0)

Table 9: Done rows are dated from the release tag that first contains the implementing commit; the four-component tags (v0.5.8.1 to v0.5.8.4 and the v0.5.5.x/v0.5.6.x line) are point releases that `CHANGELOG.md` groups under its published [0.5.5], [0.5.8] and [0.6.0-preview] entries, so look there rather than for a tag-numbered entry. Planned rows follow the repository roadmap; each was confirmed absent from `src-tauri/src` and `src` before being listed — there is no offline solve path (`solve_offline_placeholder` always fails), no command exposes `generate_star_mask`, no undo or action log exists beside the fixed `ProcessingChain`, and local normalization, mosaic assembly, comet alignment and the RAW/XISF/TIFF readers have no implementation at all. MAST likewise has no implementation: the matches for “mast” in the tree are `CalibrationMasters` and master bias/dark/flat identifiers, plus `C:/MAST/...` fixture paths in a frontend test.

A Sample Test Data

HST/WFPC2 narrowband FITS in `exampleFits/sample-data/`: `502nmos.fits` ([OIII] 502 nm, Blue), `656nmos.fits` (H α 656 nm, Green), `673nmos.fits` ([SII] 673 nm, Red). 1600 $\times$ 1600 float32, WCS TAN. Public domain (NASA/ESA). The same directory carries `synthetic_catalog.csv`, the ground-truth source list emitted by the synthetic data generator, which is what photometry and alignment are validated against.